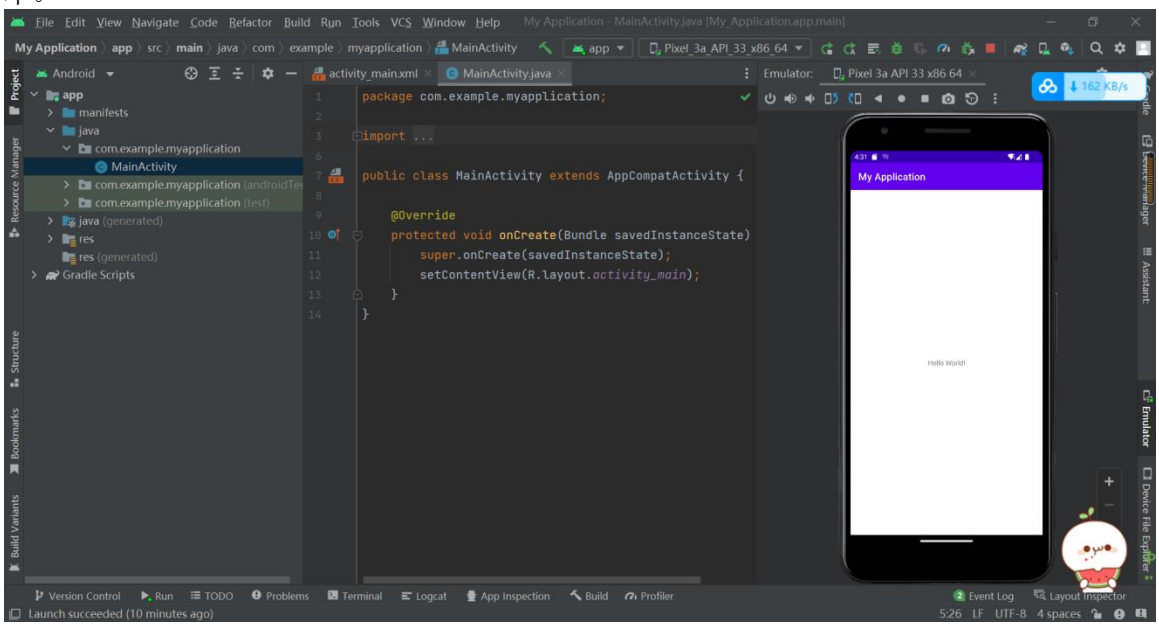
第一章作业：

P22 1. 下载最新版本的 Android Studio，安装后如果需要更新 Gradle，则将其更新到最新版本。



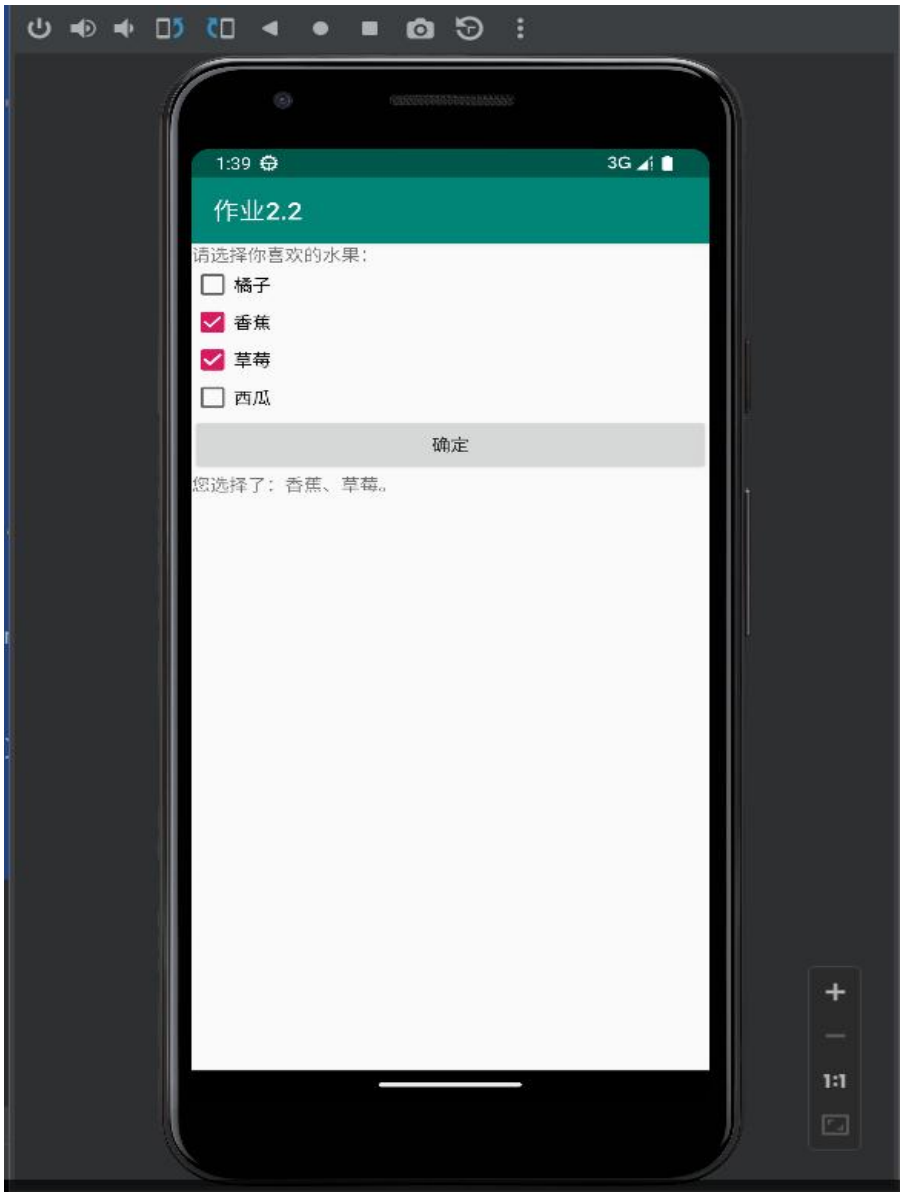
P22 2. 创建一个任意类型的手机 App 工程，分别使用虚拟设备和物理设备运行。

第二章作业：

P73 2.2 实现多选题的界面效果。

```
public class MainActivity extends AppCompatActivity implements CompoundButton.OnCheckedChan
geListener {
   private final ArrayList<String> mHobbies = new ArrayList<>();

   @SuppressLint("SetTextI18n")
   @Override
   protected void onCreate(Bundle savedInstanceState) {
      super.onCreate(savedInstanceState);
      setContentView(R.layout.activity_main);

      final TextView resultTextView = findViewById(R.id.text_view_result);
      CheckBox checkBox1 = findViewById(R.id.check_box1);
      CheckBox checkBox2 = findViewById(R.id.check_box2);
      CheckBox checkBox3 = findViewById(R.id.check_box3);
      CheckBox checkBox4 = findViewById(R.id.check_box4);
      Button submitButton = findViewById(R.id.button_submit);
      //CheckBox 设置监听器
      checkBox1.setOnCheckedChangeListener(this);
      checkBox2.setOnCheckedChangeListener(this);
      checkBox3.setOnCheckedChangeListener(this);
      checkBox4.setOnCheckedChangeListener(this);
      //Button 设置监听器
      submitButton.setOnClickListener(v -> {
         StringBuilder sb = new StringBuilder();
         for (int i = 0; i < mHobbies.size(); i++) {
```

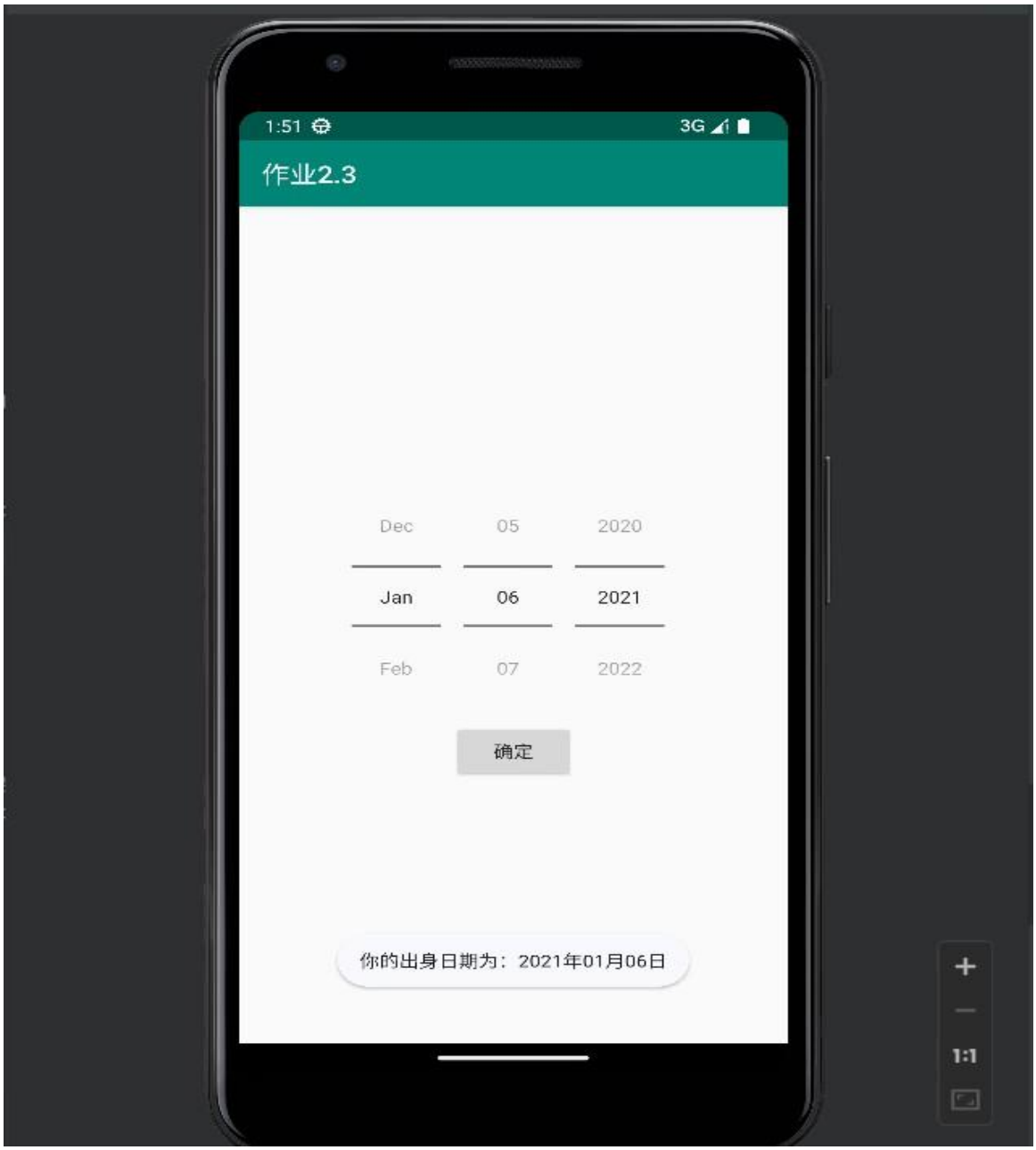
```
            //选择的兴趣爱好添加到 StringBuilder 尾部
            if (i == (mHobbies.size() - 1)) {
                sb.append(mHobbies.get(i));
            } else {
                sb.append(mHobbies.get(i)).append("、");
            }
        }
        //显示选择结果
        if (sb.length() == 0) {
            resultTextView.setText("您还没有进行选择了！");
        } else {
            resultTextView.setText("您选择了：" + sb + "。");
        }
    });
}
        // 提 示 '@param compoundButton' tag description is missing , '@param isChecked' tag description is missing
// 自动 fix

/**
 * 实现改变选项的接口方法
 *
 * @param compoundButton
 * @param isChecked
 * @return void
 */
@Override
public void onCheckedChanged(CompoundButton compoundButton, boolean isChecked) {
    if (isChecked) {
        //添加到数组
        mHobbies.add(compoundButton.getText().toString().trim());
    } else {
        //从数组中移除
        mHobbies.remove(compoundButton.getText().toString().trim());
    }
}
}
```



p73　2.3　实现选择出生日期的界面效果。

```
class MainActivity extends AppCompatActivity {
   @Override
   protected void onCreate(Bundle savedInstanceState) {
      super.onCreate(savedInstanceState);
      setContentView(R.layout.activity_main);
      final DatePicker datePicker = findViewById(R.id.date_picker);
      datePicker.updateDate(2021, 0, 1);
      datePicker.setOnDateChangedListener((view, year, monthOfYear, dayOfMonth) -> {
         Calendar calendar = Calendar.getInstance();
         calendar.set(year, monthOfYear, dayOfMonth);
          @SuppressLint("SimpleDateFormat") SimpleDateFormat simpleDateFormat = new SimpleDate
Format("yyyy 年 MM 月 dd 日");
                              Toast.makeText(getApplicationContext(), " 你 的 出 身 日 期 为 ：
" + simpleDateFormat.format(calendar.getTime()), Toast.LENGTH_SHORT).show();
```

```
});
Button button2 = findViewById(R.id.button2);
button2.setOnClickListener(v -> {
   Calendar calendar = Calendar.getInstance();
   calendar.set(datePicker.getYear(), datePicker.getMonth(), datePicker.getDayOfMonth());
    @SuppressLint("SimpleDateFormat") SimpleDateFormat simpleDateFormat = new SimpleDate
Format("yyyy 年 MM 月 dd 日");
                    Toast.makeText(getApplicationContext(), " 你 的 出 身 日 期 为 ：
" + simpleDateFormat.format(calendar.getTime()), Toast.LENGTH_SHORT).show(); });
```



第三章：

p87 3.1 实现注册界面的布局效果，至少包括用户名、密码、忘记密码、登录等控件。

```
<TableLayout xmlns:android="http://schemas.android.com/apk/res/android"
  android:layout_width="300dp"
  android:layout_height="wrap_content"
  android:layout_gravity="center"
  android:collapseColumns="2"
  android:stretchColumns="1">
  <!--第 1 行-->
  <TableRow>
    <!--第 1 列-->
    <TextView
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="用户名" />
    <!--第 2 列-->
    <EditText
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:ems="10"
      android:inputType="textPersonName" />
  </TableRow>
  <!--第 2 行-->
  <TableRow>
    <!--第 1 列-->
    <TextView
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="密码" />
    <!--第 2 列-->
    <EditText
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:ems="10"
      android:inputType="textPassword" />
    <!--第 3 列-->
    <TextView
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="忘记密码-+" />
  </TableRow>
  <!--第 3 行-->
  <TableRow>
    <!--第 2 列-->
    <TextView
      android:id="@+id/textView4"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:layout_column="1"
      android:gravity="end"
      android:text="注册账号" />
```

```
    </TableRow>
    <!--第 4 行-->
    <TableRow>
        <!--第 1 列和第 2 列合并-->
        <Button
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_span="2"
            android:text="登录" />
    </TableRow>
</TableLayout>

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }}
```


2:33
作业3.1
用户名 123123
密码 ••••••••
忘记密码
登录

第四章：
p122 4.3 实现京东首页的轮播广告效果，至少包括 3 个产品广告的图像。

```
public class SlideShowView extends FrameLayout {

    // 使用 universal-image-loader 插件读取网络图片，需要工程导入 universal-image-loader-1.8.6-with-sources.jar
    private ImageLoader imageLoader = ImageLoader.getInstance();

  //轮播图图片数量
  private final static int IMAGE_COUNT = 5;
  //自动轮播的时间间隔
  private final static int TIME_INTERVAL = 5;
  //自动轮播启用开关
  private final static boolean isAutoPlay = true;

  //自定义轮播图的资源
  private String[] imageUrls;
  //放轮播图片的 ImageView 的 list
  private List<ImageView> imageViewsList;
  //放圆点的 View 的 list
  private List<View> dotViewsList;

  private ViewPager viewPager;
  //当前轮播页
  private int currentItem  = 0;
  //定时任务
  private ScheduledExecutorService scheduledExecutorService;

  private Context context;

  //Handler
  private Handler handler = new Handler(){

    @Override
    public void handleMessage(Message msg) {
      // TODO Auto-generated method stub
      super.handleMessage(msg);
      viewPager.setCurrentItem(currentItem);
    }

  };

  public SlideShowView(Context context) {
    this(context,null);
    // TODO Auto-generated constructor stub
  }
  public SlideShowView(Context context, AttributeSet attrs) {
    this(context, attrs, 0);
    // TODO Auto-generated constructor stub
```

```
}
public SlideShowView(Context context, AttributeSet attrs, int defStyle) {
   super(context, attrs, defStyle);
   this.context = context;

            initImageLoader(context);

   initData();
   if(isAutoPlay){
      startPlay();
   }

}
/**
 * 开始轮播图切换
 */
private void startPlay(){
   scheduledExecutorService = Executors.newSingleThreadScheduledExecutor();
   scheduledExecutorService.scheduleAtFixedRate(new SlideShowTask(), 1, 4, TimeUnit.SECONDS);
}
/**
 * 停止轮播图切换
 */
private void stopPlay(){
   scheduledExecutorService.shutdown();
}
/**
 * 初始化相关 Data
 */
private void initData(){
   imageViewsList = new ArrayList<ImageView>();
   dotViewsList = new ArrayList<View>();

   // 一步任务获取图片
   new GetListTask().execute("");
}
/**
 * 初始化 Views 等 UI
 */
private void initUI(Context context){
     if(imageUrls == null || imageUrls.length == 0)
            return;

   LayoutInflater.from(context).inflate(R.layout.layout_slideshow, this, true);

   LinearLayout dotLayout = (LinearLayout)findViewById(R.id.dotLayout);
   dotLayout.removeAllViews();

   // 热点个数与图片特殊相等
   for (int i = 0; i < imageUrls.length; i++) {
```

```
        ImageView view =  new ImageView(context);
        view.setTag(imageUrls[i]);
        if(i==0)//给一个默认图
                view.setBackgroundResource(R.drawable.appmain_subject_1);
        view.setScaleType(ScaleType.FIT_XY);
        imageViewsList.add(view);

        ImageView dotView =  new ImageView(context);
        LinearLayout.LayoutParams params = new
LinearLayout.LayoutParams(LayoutParams.WRAP_CONTENT,LayoutParams.WRAP_CONTENT);
        params.leftMargin = 4;
                        params.rightMargin = 4;
                        dotLayout.addView(dotView, params);
        dotViewsList.add(dotView);
                }

      viewPager = (ViewPager) findViewById(R.id.viewPager);
      viewPager.setFocusable(true);

      viewPager.setAdapter(new MyPagerAdapter());
      viewPager.setOnPageChangeListener(new MyPageChangeListener());
    }

    /**
     * 填充 ViewPager 的页面适配器
     *
     */
    private class MyPagerAdapter  extends PagerAdapter{

      @Override
      public void destroyItem(View container, int position, Object object) {
        // TODO Auto-generated method stub
        //((ViewPag.er)container).removeView((View)object);
        ((ViewPager)container).removeView(imageViewsList.get(position));
      }

      @Override
      public Object instantiateItem(View container, int position) {
        ImageView imageView = imageViewsList.get(position);

                        imageLoader.displayImage(imageView.getTag() + "", imageView);

        ((ViewPager)container).addView(imageViewsList.get(position));
        return imageViewsList.get(position);
      }

      @Override
      public int getCount() {
        // TODO Auto-generated method stub
        return imageViewsList.size();
```

```
    }

    @Override
    public boolean isViewFromObject(View arg0, Object arg1) {
        // TODO Auto-generated method stub
        return arg0 == arg1;
    }
    @Override
    public void restoreState(Parcelable arg0, ClassLoader arg1) {
        // TODO Auto-generated method stub

    }

    @Override
    public Parcelable saveState() {
        // TODO Auto-generated method stub
        return null;
    }

    @Override
    public void startUpdate(View arg0) {
        // TODO Auto-generated method stub

    }

    @Override
    public void finishUpdate(View arg0) {
        // TODO Auto-generated method stub

    }

}
/**
 * ViewPager 的监听器
 * 当 ViewPager 中页面的状态发生改变时调用
 *
 */
private class MyPageChangeListener implements OnPageChangeListener{

    boolean isAutoPlay = false;

    @Override
    public void onPageScrollStateChanged(int arg0) {
        // TODO Auto-generated method stub
        switch (arg0) {
        case 1:// 手势滑动，空闲中
            isAutoPlay = false;
            break;
        case 2:// 界面切换中
            isAutoPlay = true;
```

```
            break;
        case 0:// 滑动结束，即切换完毕或者加载完毕
            // 当前为最后一张，此时从右向左滑，则切换到第一张
            if (viewPager.getCurrentItem() == viewPager.getAdapter().getCount() - 1 && !isAutoPlay) {
                viewPager.setCurrentItem(0);
            }
            // 当前为第一张，此时从左向右滑，则切换到最后一张
            else if (viewPager.getCurrentItem() == 0 && !isAutoPlay) {
                viewPager.setCurrentItem(viewPager.getAdapter().getCount() - 1);
            }
            break;
    }
    }

    @Override
    public void onPageScrolled(int arg0, float arg1, int arg2) {
        // TODO Auto-generated method stub

    }

    @Override
    public void onPageSelected(int pos) {
        // TODO Auto-generated method stub

        currentItem = pos;
        for(int i=0;i < dotViewsList.size();i++){
            if(i == pos){
                ((View)dotViewsList.get(pos)).setBackgroundResource(R.drawable.dot_focus);
            }else {
                ((View)dotViewsList.get(i)).setBackgroundResource(R.drawable.dot_blur);
            }
        }
    }

}

/**
 *执行轮播图切换任务
 *
 */
private class SlideShowTask implements Runnable{

    @Override
    public void run() {
        // TODO Auto-generated method stub
        synchronized (viewPager) {
            currentItem = (currentItem+1)%imageViewsList.size();
            handler.obtainMessage().sendToTarget();
        }
    }
```

```
}

/**
 * 销毁 ImageView 资源，回收内存
 *
 */
private void destoryBitmaps() {

    for (int i = 0; i < IMAGE_COUNT; i++) {
        ImageView imageView = imageViewsList.get(i);
        Drawable drawable = imageView.getDrawable();
        if (drawable != null) {
            //解除 drawable 对 view 的引用
            drawable.setCallback(null);
        }
    }
}


    /**
     * 异步任务,获取数据
     *
     */
    class GetListTask extends AsyncTask<String, Integer, Boolean> {

        @Override
        protected Boolean doInBackground(String... params) {
            try {
                // 这里一般调用服务端接口获取一组轮播图片，下面是从百度找的几个图
片

                imageUrls = new String[]{
                        "http://image.zcool.com.cn/56/35/1303967876491.jpg",
                        "http://image.zcool.com.cn/59/54/m_1303967870670.jpg",
                        "http://image.zcool.com.cn/47/19/1280115949992.jpg",
                        "http://image.zcool.com.cn/59/11/m_1303967844788.jpg"
                };
                return true;
            } catch (Exception e) {
                e.printStackTrace();
                return false;
            }
        }
        @Override
        protected void onPostExecute(Boolean result) {
            super.onPostExecute(result);
            if (result) {
           initUI(context);
            }}}
```

```
	/**
	 * ImageLoader 图片组件初始化
	 *
	 * @param context
	 */
	public static void initImageLoader(Context context) {
		// This configuration tuning is custom. You can tune every option, you
		// may tune some of them,
		// or you can create default configuration by
		// ImageLoaderConfiguration.createDefault(this);
		// method.
		ImageLoaderConfiguration config = new
ImageLoaderConfiguration.Builder(context).threadPriority(Thread.NORM_PRIORITY -
2).denyCacheImageMultipleSizesInMemory().discCacheFileNameGenerator(new
Md5FileNameGenerator()).tasksProcessingOrder(QueueProcessingType.LIFO).writeDebugLogs() //
Remove
// for
// release
// app
.build();
// Initialize ImageLoader with configuration.
ImageLoader.getInstance().init(config);
	}
```


19:58
BabyShop
热血屌丝的反杀

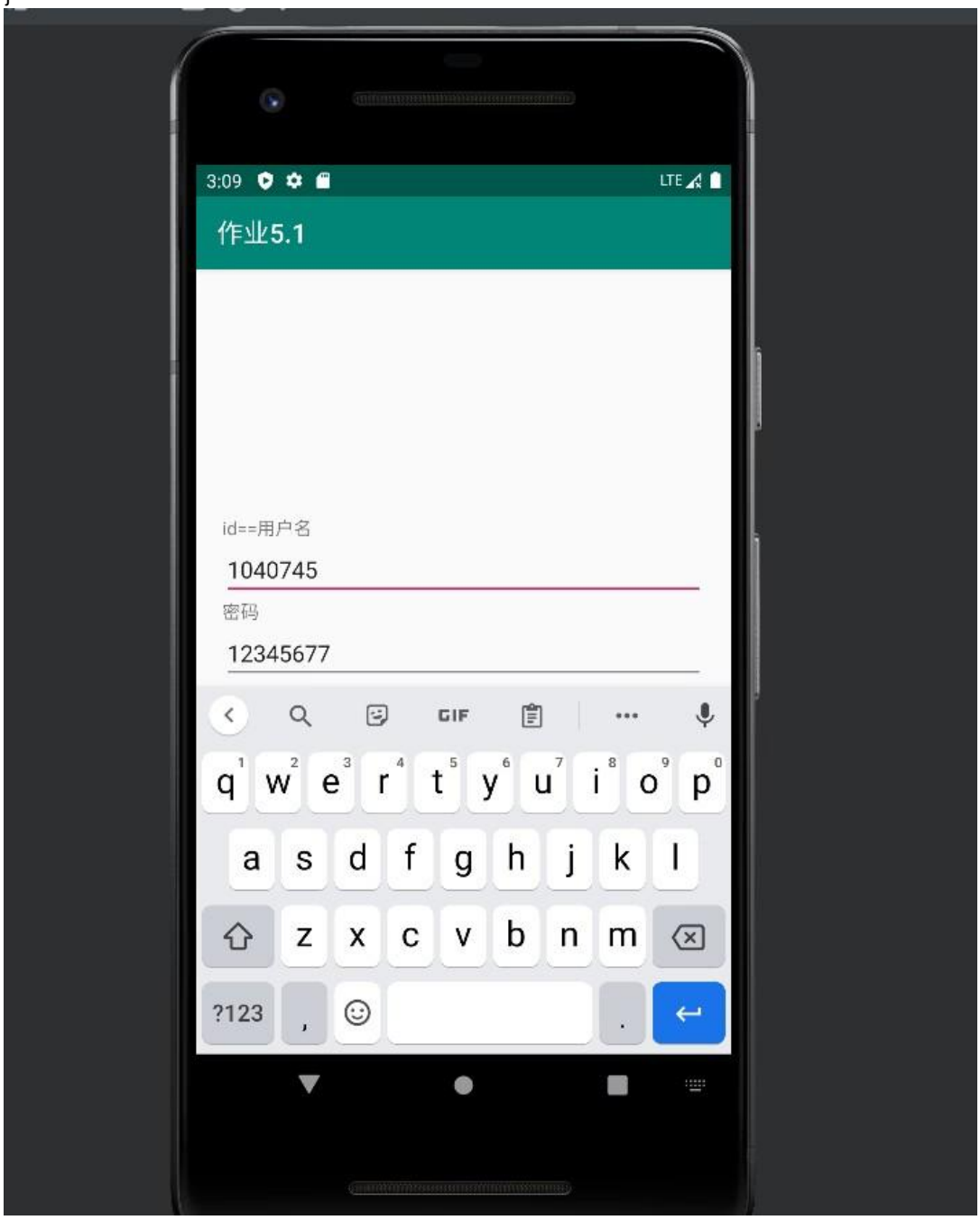

时光会紧锁，美梦不曾停
Blazing times

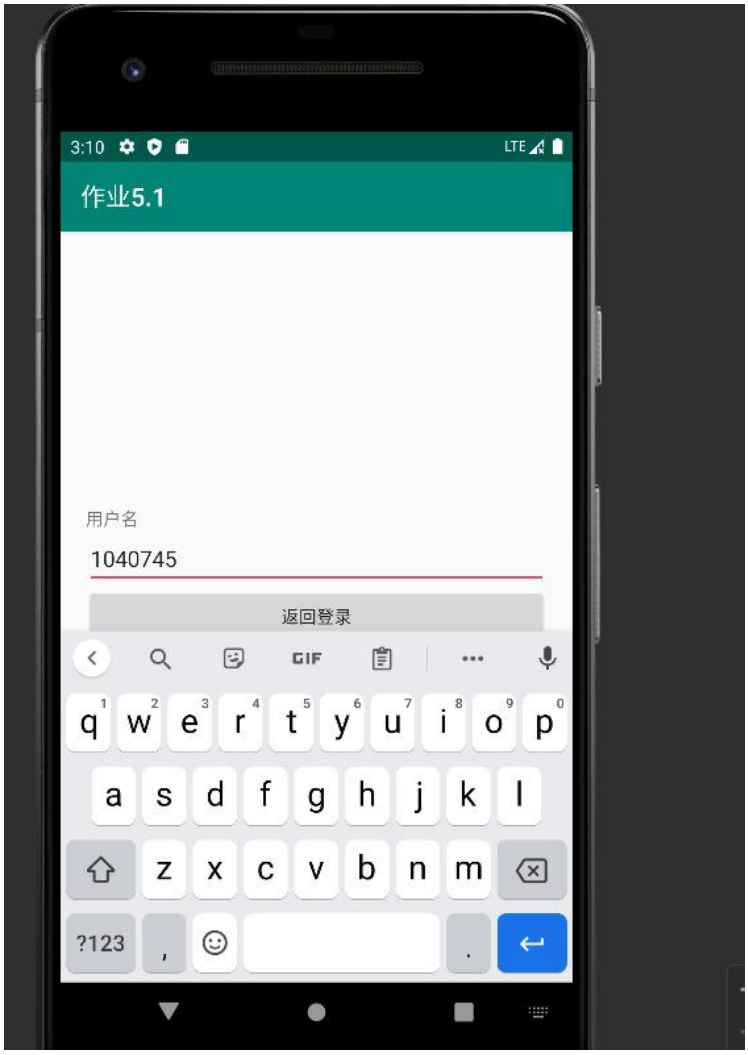
第五章：

p150 5.1 实现登录后显示用户名的效果。输入用户名、密码后登录，启动一个新的 Activity 显示用户名。

```
public class MainActivity extends AppCompatActivity {
    private static final int REQUEST_CODE = 101;
    private EditText mResultEditText;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        mResultEditText = findViewById(R.id.edit_text_result);
        //发送数据（没有回传数据）
        findViewById(R.id.button_send).setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                Intent intent = new Intent(MainActivity.this, ReceiveActivity.class);
                intent.putExtra("用户名", );
                intent.putExtra("密码");
                startActivity(intent);
                mResultEditText.setText("");
            }
        });
        //发送数据（接收回传数据）
        findViewById(R.id.button_send_for_result).setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                Intent intent = new Intent(MainActivity.this, ReceiveActivity.class);
                intent.putExtra("用户名");
                intent.putExtra("密码");
                startActivityForResult(intent, REQUEST_CODE);
                mResultEditText.setText("");
            }
        });
    }
    //处理回调数据
    protected void onActivityResult(int requestCode, int resultCode, Intent data) {
        super.onActivityResult(requestCode, resultCode, data);
        mResultEditText.setText("********");
        //判断请求码
        if (requestCode == REQUEST_CODE) {
            //判断结果码
            if (resultCode == ReceiveActivity.RESULT_CODE) {
                String result = data.getStringExtra("data");
                mResultEditText.setText(result);
            }else{
                mResultEditText.setText("用户名无效");
            }
        }
```

```
    }
}
```


3:09
LTE
作业5.1
id==用户名
1040745
密码
12345677
GIF
?123



第六章：

p176 6.3 通过广播接收器实现两个 App 之间通过自定义广播进行数据通信。

```
<application
    android:allowBackup="true"
    android:icon="@mipmap/ic_launcher"
    android:label="@string/app_name"
    android:roundIcon="@mipmap/ic_launcher_round"
    android:supportsRtl="true"
    android:theme="@style/Theme.T604">
    <receiver
        android:name=".MyReceiver"
        android:enabled="true"
        android:exported="true">
        <intent-filter>
            <action android:name="MyRecceiver.Custom"/>
        </intent-filter>
    </receiver>
    <activity android:name=".MainActivity">
        <intent-filter>
```

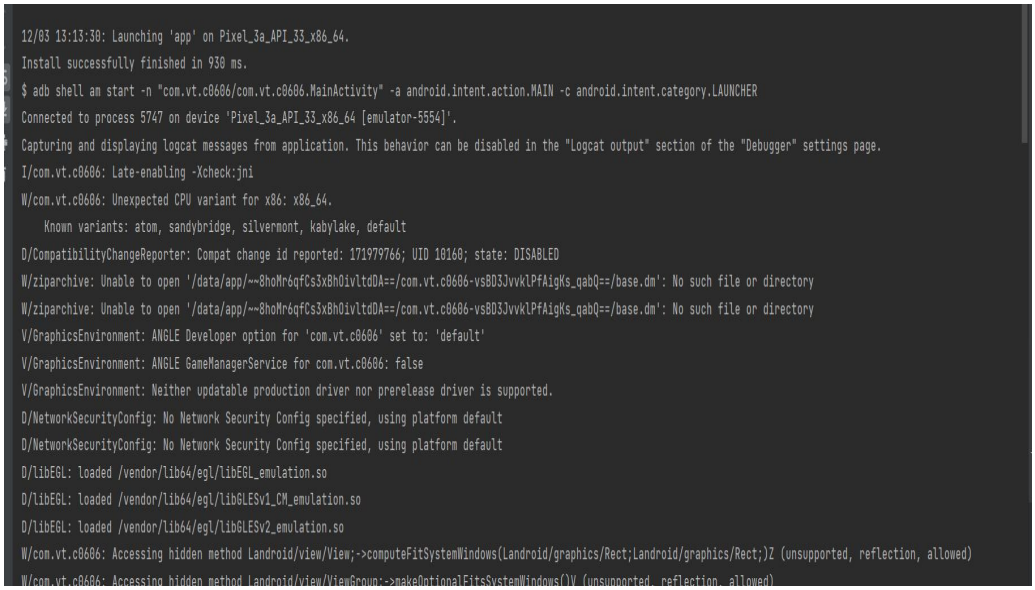
```
            <action android:name="android.intent.action.MAIN" />

            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
      </activity>
  </application>
</manifest>
```

```
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
  <application
      android:allowBackup="true"
      android:icon="@mipmap/ic_launcher"
      android:label="@string/app_name"
      android:roundIcon="@mipmap/ic_launcher_round"
      android:supportsRtl="true"
      android:theme="@style/Theme.T603">
      <receiver
          android:name=".MyReceiver"
          android:enabled="true"
          android:exported="true">
          <intent-filter>
              <action android:name=" MyRecceiver.Custom"/>
          </intent-filter>
      </receiver>
      <activity android:name=".MainActivity">
          <intent-filter>
              <action android:name="android.intent.action.MAIN" />

              <category android:name="android.intent.category.LAUNCHER" />
          </intent-filter>
      </activity>
  </application>
</manifest>
```

```
private MyReceiver myReceiver = new MyReceiver();
private boolean mRegistered = false;
@Override
protected void onCreate(Bundle savedInstanceState) {
  super.onCreate(savedInstanceState);
  setContentView(R.layout.activity_main);
  findViewById(R.id.btn_send1).setOnClickListener(v->{
        Intent intent=new Intent();
        intent.setAction("MyRecceiver.Custom");
        intent.putExtra("info","玛卡巴卡，古西蒂希（T603->T604）");
        intent.addFlags(Intent.FLAG_ACTIVITY_PREVIOUS_IS_TOP);
        sendBroadcast(intent);
```

```
    });
    findViewById(R.id.btn_register).setOnClickListener(v -> {
        IntentFilter intentFilter = new IntentFilter();
     intentFilter.addAction(ConnectivityManager.CONNECTIVITY_ACTION);
        intentFilter.addAction(Intent.ACTION_SCREEN_ON);
        intentFilter.addAction(Intent.ACTION_SCREEN_OFF);
        intentFilter.addAction(Intent.ACTION_USER_PRESENT);
        intentFilter.addAction(Intent.ACTION_BATTERY_OKAY);
        intentFilter.addAction(Intent.ACTION_BATTERY_LOW);
        intentFilter.addAction(Intent.ACTION_BATTERY_CHANGED);
        intentFilter.addAction("com.vt.t604.MyReceiver.ss");
        registerReceiver(myReceiver, intentFilter);
        mRegistered = true;
    });
    findViewById(R.id.btn_send2).setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View v) {
            Intent intent=new Intent();
            intent.setAction("MyRecceiver.Custom");
            intent.putExtra("info","广告信息已发送");
            intent.addFlags(Intent.FLAG_ACTIVITY_PREVIOUS_IS_TOP);
            sendOrderedBroadcast(intent,null);
        }
    });
}
```


5:13
3G
作业6.3
显示手机状况
发送第一次广播
发送第二次广播



第七章：

p213 7.2 使用 SQLite 实现记录登录时间的功能，并且能够查询登录的历史记录。

```
public class HistoryActivity extends AppCompatActivity implements View.OnClickListener {
   private Button cancel;
   public EditText mIdEditText;
   public EditText mNameEditText;
   public EditText mPhoneEditText;
   private Context mContext;
   private SQLiteDatabase mSQLiteDataBase;
   private ContactSQLiteOpenHelper mContactSQLiteOpenHelper;
   private String mDataBaseName = "contacts.db";
   private String mTableName = "contacts";
   private List<ContactModel> mContactModel;
   private ContactListAdapter mContactListAdapter;
   private ListView mContactListView;
   @Override
   protected void onCreate(Bundle savedInstanceState) {
      setContentView(R.layout.activity_history);
      super.onCreate(savedInstanceState);
     this.setTitle("登录历史");
      mContext = this;
      mContactModel = new ArrayList<>();
      mContactListView = findViewById(R.id.list_view_contacts);
 //findViewById(R.id.btn_list).setOnClickListener(this);
      init();
      cancel = (Button) findViewById(R.id.btn_cancel);
    findViewById(R.id.btn_cancel).setOnClickListener(new View.OnClickListener(){
        public void onClick(View view) {
           finish();
        }
      });
   }
   // 初始化
```

```
    private void init(){
        mContactSQLiteOpenHelper = new ContactSQLiteOpenHelper(mContext, mDataBaseName, null,
1);
        mSQLiteDataBase = mContactSQLiteOpenHelper.getWritableDatabase();
        mContactListAdapter = new ContactListAdapter(this, mContactModel);
        mContactListView.setAdapter(mContactListAdapter);
        listAllContacts();
    }
    // 显示所有联系人
    private void listAllContacts() {
        mContactModel.clear();
        Cursor cursor = mSQLiteDataBase.rawQuery("SELECT * FROM " + mTableName + " order by id asc
", null);
        while (cursor.moveToNext()) { String id = cursor.getString(cursor.getColumnIndex("id"));
 String name = cursor.getString(cursor.getColumnIndex("name"));
String phone = cursor.getString(cursor.getColumnIndex("phone"));
 mContactModel.add(new ContactModel(id, name, phone));
        }
        cursor.close();
        // 更新 ListView
        mContactListAdapter.notifyDataSetChanged();
    }
    @Override
    public void onClick(View v) {
        String id = mIdEditText.getText().toString();
        String name = mNameEditText.getText().toString();
        String phone = mPhoneEditText.getText().toString();
        switch (v.getId()) {
        }
    }
}

public class ContactListAdapter extends BaseAdapter {
    private HistoryActivity mContext;
    private List<ContactModel> mContactModel;
    private ViewHolder mViewHolder;
    //构造方法
    public ContactListAdapter(HistoryActivity mContext, List<ContactModel> mContactModel) {
        this.mContext = mContext;
        this.mContactModel = mContactModel;}
    @Override
    public int getCount() {
        return mContactModel.size(); }
    @Override
    public Object getItem(int position) {
        return mContactModel;}
    @Override
    public long getItemId(int position) {
        return position;}
    //获取列表项视图
```

```
  @Override
  public View getView(int position, View convertView, ViewGroup parent) {
    if (convertView == null) {//convertView 未实例化时
      convertView = LayoutInflater.from(mContext).inflate(R.layout.list_view_item_contact, parent, f
alse);
      mViewHolder = new ViewHolder();
      mViewHolder.idTextView = convertView.findViewById(R.id.text_view_id);
      mViewHolder.nameTextView = convertView.findViewById(R.id.text_view_name);
      mViewHolder.phoneTextView = convertView.findViewById(R.id.text_view_phone);
      convertView.setTag(mViewHolder);//holder 缓存到 convertView
    } else {//convertView 实例化时
          mViewHolder = (ViewHolder) convertView.getTag();// 从 convertView 获取 holder 缓
存  mViewHolder.idTextView.setText(mContactModel.get(position).id);
    mViewHolder.idTextView.setId(position);
     mViewHolder.idTextView.setOnClickListener(mTextViewOnClickListener);    mViewHolder.nameT
extView.setText(mContactModel.get(position).name);
    mViewHolder.nameTextView.setId(position);  mViewHolder.nameTextView.setOnClickListener(mT
extViewOnClickListener);      mViewHolder.phoneTextView.setText(mContactModel.get(position).pho
ne);
    mViewHolder.phoneTextView.setId(position); mViewHolder.phoneTextView.setOnClickListener(m
TextViewOnClickListener);
    return convertView;
  }
   private View.OnClickListener mTextViewOnClickListener = new View.OnClickListener() {
    public void onClick(View v) {    (mContext.mIdEditText).setText(mContactModel.get(v.getId()).id);
      (mContext.mNameEditText).setText(mContactModel.get(v.getId()).name);      (mContext.mPh
oneEditText).setText(mContactModel.get(v.getId()).phone);}};
  class ViewHolder {
    TextView idTextView;
    TextView nameTextView;
    TextView phoneTextView;}}
```

```
public class ContactSQLiteOpenHelper extends SQLiteOpenHelper {
   public ContactSQLiteOpenHelper(Context context, String name, SQLiteDatabase.CursorFactory fact
ory, int version){
    super(context, name, factory, version);}
  @Override
  public void onCreate(SQLiteDatabase db) {
    // 创建 contacts 表
    db.execSQL("CREATE TABLE contacts(id INTEGER PRIMARY KEY AUTOINCREMENT,name VARCHAR
(16),phone VARCHAR(11))");}
  @Override
  public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
    if (newVersion > oldVersion) {
      // contacts 表添加 notes 列
      db.execSQL("ALTER TABLE contacts ADD notes VARCHAR(20)"); }}}
```


Emulator:
Pixel 3a API 33 x86 64
6:00
3G
登录界面
账号
110
密码
注册
登录
请注意密码大小写
1
2 ABC
3 DEF
4 GHI
5 JKL
6 MNO
7 PQRS
8 TUV
9 WXYZ
* #
0 +
.
1:1

第八章：

p271 8.1 使用 Camera2 实现定时拍照的功能。

```
private Runnable timerRunnable = new Runnable() {
    @Override
    public void run() {
      if (mCurrentTimer > 0) {
        mTvCountDown.setText(mCurrentTimer + "");

        mCurrentTimer--;
        mHandler.postDelayed(timerRunnable, 1000);
```

```
} else {
    mTvCountDown.setText("");

    mCamera.takePicture(null, null, null, MainActivity.this);
    playSound();

    mIsTimerRunning = false;
    mCurrentTimer = 5;
}    }    }
```